In [ ]:
```
import marimo as mo
```

In [ ]:
```
import z3

print("Z3 disponível")
print(z3.get_version_string())
```

TP2 - Conway's Game of Life

Introdução

O Conway's Game of Life é um autómato celular no qual cada célula de uma grelha pode estar viva ou morta. A evolução do sistema é determinada pelo estado das células vizinhas, seguindo regras de nascimento, sobrevivência e morte.

Neste trabalho é considerada uma versão modificada do autómato, com uma grelha de dimensão finita e células adicionais nas fronteiras, cujos valores permanecem constantes.

O problema será modelado através de uma Máquina de Estados Finita (FSM), permitindo representar a evolução do sistema e verificar propriedades do seu comportamento.

Objetivo

O objetivo deste trabalho é construir uma FSM que represente o funcionamento do Conway's Game of Life modificado, considerando duas modalidades de transição:

- **Atualização individual:** em cada transição é atualizada uma única célula normal.
- **Atualização simultânea:** em cada transição são atualizadas todas as células normais.

Pretende-se também verificar duas propriedades:

1. Se o sistema pode alcançar um estado em que todas as células normais estão mortas.
2. Se alguma célula normal pode permanecer definitivamente viva ou morta.

A verificação deverá permitir demonstrar as propriedades ou apresentar contraexemplos quando estas não se verificam.

Abordagem

A implementação será desenvolvida em Python, utilizando o Z3 para representar simbolicamente os estados e as relações de transição do autómato.

Cada célula normal será representada por uma variável booleana. As células adicionais das fronteiras terão valores constantes durante a evolução do sistema.

A FSM será definida através de:

- Um conjunto de variáveis que representam os estados das células.
- Um predicado que caracteriza os estados iniciais.
- Uma relação de transição que determina as mudanças de estado segundo a regra B3/S23.

As duas modalidades de transição serão selecionadas através de um parâmetro.

Para analisar as propriedades pedidas, serão utilizadas técnicas de verificação de modelos baseadas em SAT/SMT, permitindo procurar estados alcançáveis que violem as propriedades e, quando possível, demonstrar a sua validade.

A1 - Representação dos estados

O espaço de estados do autómato é constituído por uma grelha de dimensão `N × N`, com `N > 16`, onde cada célula normal pode assumir um de dois valores:

- `0` — célula morta;
- `1` — célula viva.

No estado inicial, todas as células normais estão mortas, exceto as que pertencem ao bloco `3 × 3` situado no canto superior esquerdo.

Para representar a configuração inicial é utilizada uma matriz de listas Python. A dimensão `N` é parametrizável, começando-se com `N = 17`, o menor valor permitido pelo enunciado.

Na implementação, os índices das listas começam em `0`, pelo que as células inicialmente vivas correspondem às posições `0 ≤ i < 3` e `0 ≤ j < 3`.

In [ ]:
```
N = 17

grelha = [
    [1 if i < 3 and j < 3 else 0 for j in range(N)]
    for i in range(N)
]

for linha in grelha:
    print(" ".join(map(str, linha)))
```

Teste da configuração inicial

É verificado que:

- a grelha possui exatamente `N` linhas e `N` colunas;
- existem exatamente 9 células inicialmente vivas;
- todas as restantes células normais estão mortas.

Este teste permite confirmar que a configuração inicial respeita as condições definidas para o autómato.

In [ ]:
```python
assert len(grelha) == N
assert all(len(linha) == N for linha in grelha)

total_vivas = sum(sum(linha) for linha in grelha)

assert total_vivas == 9

for i in range(N):
    for j in range(N):
        esperado = 1 if i < 3 and j < 3 else 0
        assert grelha[i][j] == esperado

print("Configuração inicial validada.")
print("Dimensão:", N, "x", N)
print("Células vivas:", total_vivas)
print("Células mortas:", N * N - total_vivas)
```

A2 — Células de fronteira

Além das `N²` células normais, o autómato possui dois conjuntos de células adicionais:

- **Células iniciais:** `2N + 1` células situadas na linha `0` ou na coluna `0`. Os seus valores são gerados aleatoriamente, com probabilidade `1/2` de estarem vivas, e permanecem constantes durante a evolução.
- **Células finais:** `2N + 3` células situadas na linha `N + 1` ou na coluna `N + 1`, excluindo as células iniciais. Estas permanecem sempre mortas.

As células de fronteira influenciam o número de vizinhos vivos das células normais próximas das extremidades.

Para garantir a reprodução dos testes, será utilizada uma semente fixa na geração aleatória das células iniciais.

In [ ]:
```python
import random

rng = random.Random(42)

iniciais = {
    (i, j): rng.randint(0, 1)
    for i in range(N + 1)
    for j in range(N + 1)
    if i == 0 or j == 0
}

finais = {
    (i, j): 0
    for i in range(N + 2)
```

```
    for j in range(N + 2)
    if i == N + 1 or j == N + 1
}

print("Células iniciais:", len(iniciais))
print("Células finais:", len(finais))
```

Teste das células de fronteira

É verificado que:

- existem exatamente `2N + 1` células iniciais;
- existem exatamente `2N + 3` células finais;
- todas as células finais estão mortas;
- nenhuma célula pertence simultaneamente aos dois conjuntos.

A geração aleatória das células iniciais utiliza uma semente fixa para permitir reproduzir os resultados.

In [ ]:
```
assert len(iniciais) == 2 * N + 1
assert len(finais) == 2 * N + 3

assert all(valor == 0 for valor in finais.values())

assert set(iniciais).isdisjoint(set(finais))

assert all(valor in (0, 1) for valor in iniciais.values())

print("Células de fronteira validadas.")
```

A3 — Definição da vizinhança e regra B3/S2

Definição da vizinhança

Cada célula normal pode ter até 8 vizinhos, considerando as posições horizontais, verticais e diagonais.

Duas células `(i0, j0)` e `(i1, j1)` são consideradas vizinhas quando:

$$\max(|i_0 - i_1|, |j_0 - j_1|) = 1$$

A função `vizinhos` recebe as coordenadas de uma célula e devolve as posições dos seus 8 vizinhos.

Para obter o estado de cada vizinho, é utilizada a função `valor_celula`, que distingue entre células normais, iniciais e finais.

A função `contar_vizinhos_vivos` utiliza estas duas funções para determinar quantos vizinhos de uma célula estão vivos.

In [ ]:
```python
# Código desenvolvido com apoio de LLM.
# Implementação revista e validada através dos testes do notebook.

def vizinhos(i, j):
    return [
        (i + di, j + dj)
        for di in (-1, 0, 1)
        for dj in (-1, 0, 1)
        if (di, dj) != (0, 0)
    ]


def valor_celula(i, j, grelha, iniciais, finais):
    # Células iniciais
    if (i, j) in iniciais:
        return iniciais[(i, j)]

    # Células finais
    if (i, j) in finais:
        return finais[(i, j)]

    # Células normais
    n = len(grelha)

    if 1 <= i <= n and 1 <= j <= n:
        return grelha[i - 1][j - 1]

    raise ValueError("Coordenadas fora da grelha")


def contar_vizinhos_vivos(i, j, grelha, iniciais, finais):
```

```
    return sum(
        valor_celula(vi, vj, grelha, iniciais, finais)
        for vi, vj in vizinhos(i, j)
    )
```

Regra de evolução B3/S23

A evolução das células normais segue a regra B3/S23 do Conway's Game of Life.

O estado seguinte de cada célula depende do seu estado atual e do número de vizinhos vivos:

- **Nascimento (B3):** uma célula morta passa a viva quando possui exatamente 3 vizinhos vivos.
- **Sobrevivência (S23):** uma célula viva mantém-se viva quando possui 2 ou 3 vizinhos vivos.
- **Morte:** nos restantes casos, a célula fica morta.

A função `regra_b3_s23` implementa estas condições e devolve `1` se a célula estiver viva no estado seguinte ou `0` se estiver morta.

In [ ]:
```
def regra_b3_s23(atual, vivos):
    # Nascimento
    if atual == 0 and vivos == 3:
        return 1

    # Sobrevivência
    if atual == 1 and vivos in (2, 3):
        return 1

    # Morte ou permanência no estado morto
    return 0
```

Testes da vizinhança e da regra B3/S23

São realizados testes para verificar:

- que cada célula normal possui exatamente 8 posições vizinhas;
- que a vizinhança da célula `(1, 1)` inclui corretamente as células da fronteira;
- que a contagem de vizinhos vivos considera tanto células normais como células iniciais;
- que uma região sem células vivas apresenta uma contagem igual a zero;
- que a regra B3/S23 produz os resultados esperados para todos os números possíveis de vizinhos vivos.

Estes testes permitem validar as funções que serão utilizadas posteriormente na construção da relação de transição da FSM.

In [ ]:
```
# Testar a vizinhança
vizinhos_teste = vizinhos(1, 1)
```

```
assert len(vizinhos_teste) == 8

assert set(vizinhos_teste) == {
    (0, 0), (0, 1), (0, 2),
    (1, 0),         (1, 2),
    (2, 0), (2, 1), (2, 2)
}

print("Vizinhança validada.")


# Testar a contagem de vizinhos vivos
contagem_esperada = 3 + sum(
    iniciais[pos]
    for pos in [(0, 0), (0, 1), (0, 2), (1, 0), (2, 0)]
)

assert contar_vizinhos_vivos(
    1, 1, grelha, iniciais, finais
) == contagem_esperada

assert contar_vizinhos_vivos(
    5, 5, grelha, iniciais, finais
) == 0

print("Contagem de vizinhos vivos validada.")


# Testar todas as possibilidades da regra B3/S23
for numero_vivos in range(9):
    assert regra_b3_s23(0, numero_vivos) == int(
        numero_vivos == 3
    )

    assert regra_b3_s23(1, numero_vivos) == int(
        numero_vivos in (2, 3)
    )

print("Regra B3/S23 validada.")
```

A4 — Construção da máquina de estados finita (FSM)

Representação simbólica dos estados

O autómato é modelado através de uma Máquina de Estados Finita (FSM), definida por:

$$\Sigma = (Q, I, T)$$

onde:

- `Q` representa o espaço de estados;

- `I` representa o conjunto de estados iniciais;
- `T` representa a relação de transição entre estados.

Cada célula normal é representada por uma variável booleana do Z3, que indica se está viva (`True`) ou morta (`False`).

Para representar uma transição, são criados dois conjuntos de variáveis:

- `estado0` : valores das células antes da transição;
- `estado1` : valores das células depois da transição.

As células iniciais da fronteira são representadas por variáveis booleanas simbólicas, permitindo considerar todas as configurações iniciais possíveis.

As células finais são representadas pela constante booleana `False` , uma vez que permanecem sempre mortas.

As variáveis de fronteira são partilhadas entre os diferentes estados, garantindo que os seus valores permanecem constantes.

In [ ]:
```python
def criar_estado_z3(n, instante):
    return {
        (i, j): z3.Bool(f"x_{instante}_{i}_{j}")
        for i in range(1, n + 1)
        for j in range(1, n + 1)
    }


# Estados antes e depois de uma transição
estado0 = criar_estado_z3(N, 0)
estado1 = criar_estado_z3(N, 1)

# Fronteira inicial simbólica
borda_inicial_z3 = {
    (i, j): z3.Bool(f"b_{i}_{j}")
    for i, j in iniciais
}

# Fronteira final permanentemente morta
borda_final_z3 = {
    pos: z3.BoolVal(False)
    for pos in finais
}

print("Variáveis normais por estado:", len(estado0))
print("Variáveis da fronteira inicial:", len(borda_inicial_z3))
print("Células finais constantes:", len(borda_final_z3))
```

Definição dos estados iniciais

No estado inicial, apenas as 9 células normais situadas nas posições `1 ≤ i ≤ 3` e `1 ≤ j ≤ 3` estão vivas.

Todas as restantes células normais encontram-se mortas.

O predicado `predicado_inicial` representa estas condições através de uma conjunção de restrições booleanas.

As células da fronteira inicial não são fixadas pelo predicado, permitindo que o Z3 considere qualquer combinação dos seus valores.

Esta representação corresponde ao conjunto de configurações iniciais possíveis, independentemente de uma realização aleatória concreta.

In [ ]:
```python
def predicado_inicial(estado, n):
    restricoes = []

    for i in range(1, n + 1):
        for j in range(1, n + 1):

            if i <= 3 and j <= 3:
                restricoes.append(estado[(i, j)])

            else:
                restricoes.append(z3.Not(estado[(i, j)]))

    return z3.And(*restricoes)
```

Teste do predicado inicial

É utilizado o Z3 para verificar se existe uma atribuição de valores que satisfaz todas as condições iniciais.

Um resultado `sat` indica que o conjunto de restrições é satisfazível.

Também é verificado que as 9 células inicialmente vivas são obrigatoriamente verdadeiras e que as restantes células normais são obrigatoriamente falsas.

In [ ]:
```python
teste_inicial_z3 = z3.Solver()

teste_inicial_z3.add(
    predicado_inicial(estado0, N)
)

assert teste_inicial_z3.check() == z3.sat

_modelo_inicial = teste_inicial_z3.model()

for _i in range(1, N + 1):
    for _j in range(1, N + 1):
```

```
        _valor = _modelo_inicial.eval(
            estado0[(_i, _j)],
            model_completion=True
        )

        _esperado = _i <= 3 and _j <= 3

        assert z3.is_true(_valor) == _esperado

print("Predicado inicial validado.")
```

Relação de transição

A relação de transição determina os valores das células no estado seguinte de acordo com a regra B3/S23.

Para cada célula normal, o número de vizinhos vivos é calculado simbolicamente através do Z3.

O estado seguinte da célula é definido pela expressão:

$$x'_{i,j} \leftrightarrow (v_{i,j} = 3) \vee (x_{i,j} \wedge v_{i,j} = 2)$$

onde `v` representa o número de vizinhos vivos.

São consideradas duas modalidades de atualização:

- **Individual:** é selecionada uma célula normal para aplicar a regra B3/S23. Todas as outras mantêm o seu valor.
- **Simultânea:** todas as células normais aplicam a regra B3/S23, utilizando os valores do estado anterior.

A modalidade é determinada pelo parâmetro `modo` da função `transicao_z3`.

Na atualização individual, a variável `escolhida` identifica a célula selecionada. A seleção de uma célula não obriga a que o seu valor mude, caso a regra B3/S23 determine a manutenção do estado.

In [ ]:
```
# Código desenvolvido com apoio de LLM.
# Implementação revista e validada através dos testes do notebook.

def valor_simbolico_z3(pos, estado, borda_inicial, borda_final):

    if pos in estado:
        return estado[pos]

    if pos in borda_inicial:
        return borda_inicial[pos]

    if pos in borda_final:
        return borda_final[pos]
```

```
    raise ValueError("Coordenadas inválidas")


def regra_simbolica_z3(i, j, estado, borda_inicial, borda_final):

    vivos = z3.Sum([
        z3.If(
            valor_simbolico_z3(
                pos, estado, borda_inicial, borda_final
            ),
            1,
            0
        )
        for pos in vizinhos(i, j)
    ])

    return z3.Or(
        vivos == 3,
        z3.And(estado[(i, j)], vivos == 2)
    )


def transicao_z3(
    antes, depois, borda_inicial, borda_final,
    n, modo, escolhida=None
):

    if modo not in ("individual", "simultanea"):
        raise ValueError("Modalidade inválida")

    restricoes = []

    if modo == "individual":

        if escolhida is None:
            raise ValueError("É necessária uma célula escolhida")

        restricoes.extend([
            escolhida >= 0,
            escolhida < n * n
        ])

    for i in range(1, n + 1):
        for j in range(1, n + 1):

            novo_valor = regra_simbolica_z3(
                i, j, antes, borda_inicial, borda_final
            )

            if modo == "individual":

                indice = (i - 1) * n + (j - 1)

                valor_seguinte = z3.If(
                    escolhida == indice,
```

```
                    novo_valor,
                    antes[(i, j)]
                )

            else:
                valor_seguinte = novo_valor

            restricoes.append(
                depois[(i, j)] == valor_seguinte
            )

    return z3.And(*restricoes)
```

Testes das relações de transição

São realizados testes para as duas modalidades de transição.

Primeiro, verifica-se que existe pelo menos um estado seguinte válido a partir de um estado inicial.

Na modalidade individual, é verificado que uma célula não selecionada mantém obrigatoriamente o seu valor.

Na modalidade simultânea, é verificado que a célula `(2, 2)`, inicialmente viva e com 8 vizinhos vivos, fica obrigatoriamente morta no estado seguinte.

Estes testes utilizam o Z3 para procurar violações das restrições definidas.

In [ ]:
```
# Teste da atualização simultânea

teste_simultaneo = z3.Solver()

teste_simultaneo.add(
    predicado_inicial(estado0, N)
)

teste_simultaneo.add(
    transicao_z3(
        estado0, estado1,
        borda_inicial_z3, borda_final_z3,
        N, "simultanea"
    )
)

assert teste_simultaneo.check() == z3.sat

# A célula (2,2) tem 8 vizinhos vivos e deve morrer
teste_simultaneo.push()

teste_simultaneo.add(estado1[(2, 2)])

assert teste_simultaneo.check() == z3.unsat
```

```
teste_simultaneo.pop()

print("Atualização simultânea validada.")


# Teste da atualização individual

escolhida_teste = z3.Int("escolhida_teste")

teste_individual = z3.Solver()

teste_individual.add(
    predicado_inicial(estado0, N)
)

teste_individual.add(
    transicao_z3(
        estado0, estado1,
        borda_inicial_z3, borda_final_z3,
        N, "individual", escolhida_teste
    )
)

assert teste_individual.check() == z3.sat

# Selecionar apenas a célula (1,1)
teste_individual.push()

teste_individual.add(escolhida_teste == 0)

# Tentar alterar uma célula não selecionada
teste_individual.add(
    estado1[(2, 2)] != estado0[(2, 2)]
)

assert teste_individual.check() == z3.unsat

teste_individual.pop()

print("Atualização individual validada.")
```

B1 — Alcançabilidade do estado de erro

O enunciado define como estado de erro qualquer configuração em que todas as células normais estão mortas.

Este estado é representado pelo predicado:

$$E(s) = \bigwedge_{i=1}^{N} \bigwedge_{j=1}^{N} \neg x_{i,j}$$

Pretende-se verificar a propriedade de segurança:

$$G\neg E$$

Isto significa que, em qualquer execução do autómato, o estado de erro nunca deve ser alcançado.

Para verificar a propriedade, procura-se uma sequência de transições que parta de um estado inicial válido e termine num estado de erro.

Se essa sequência existir, constitui um contraexemplo à propriedade.

In [ ]:
```python
def predicado_erro(estado):
    return z3.And(*[
        z3.Not(celula)
        for celula in estado.values()
    ])
```

Teste do estado de erro

Antes de analisar as transições, verifica-se que o estado inicial não é um estado de erro.

O Z3 deve devolver `unsat` para a conjunção do predicado inicial com o predicado de erro, uma vez que existem obrigatoriamente 9 células normais vivas.

In [ ]:
```python
teste_erro_inicial = z3.Solver()

teste_erro_inicial.add(
    predicado_inicial(estado0, N)
)

teste_erro_inicial.add(
    predicado_erro(estado0)
)

assert teste_erro_inicial.check() == z3.unsat

print("O estado inicial não é um estado de erro.")
```

Contraexemplo — Atualização individual

Na modalidade de atualização individual, é possível alcançar o estado de erro.

Para demonstrar esta situação, considera-se uma configuração válida em que todas as células iniciais da fronteira estão mortas.

Parte-se do bloco inicial de 9 células vivas e atualizam-se sucessivamente as posições:

`(1,2), (2,1), (1,1), (2,3), (2,2), (1,3), (3,1), (3,2), (3,3)`

Nesta sequência, cada atualização provoca a morte da célula selecionada, sem alterar as restantes.

Após 9 transições, todas as células normais estão mortas, constituindo um contraexemplo à propriedade de segurança.

In [ ]:
```
sequencia_erro_individual = [
    (1, 2),
    (2, 1),
    (1, 1),
    (2, 3),
    (2, 2),
    (1, 3),
    (3, 1),
    (3, 2),
    (3, 3)
]


def simular_sequencia_individual(n, sequencia):
    # Construir o estado inicial
    estado = [
        [1 if i < 3 and j < 3 else 0 for j in range(n)]
        for i in range(n)
    ]

    # Escolher todas as células iniciais mortas
    borda_zero = {
        pos: 0 for pos in iniciais
    }

    historico = [9]

    for i, j in sequencia:
        atual = estado[i - 1][j - 1]

        vivos = contar_vizinhos_vivos(
            i, j, estado, borda_zero, finais
        )

        estado[i - 1][j - 1] = regra_b3_s23(
            atual, vivos
        )

        historico.append(
            sum(sum(linha) for linha in estado)
        )

    return estado, historico


estado_final_erro, historico_erro = (
    simular_sequencia_individual(
        N, sequencia_erro_individual
    )
)

print("Células vivas por estado:", historico_erro)
```

```
assert historico_erro == list(range(9, -1, -1))
assert all(
    valor == 0
    for linha in estado_final_erro
    for valor in linha
)

print("Contraexemplo individual validado.")
```

Verificação simbólica do contraexemplo

A sequência anterior é também verificada através do Z3.

São criadas 10 configurações simbólicas, correspondentes ao estado inicial e aos estados após cada uma das 9 transições.

O solver recebe as seguintes restrições:

- O primeiro estado satisfaz o predicado inicial.
- Todas as células iniciais da fronteira estão mortas.
- Cada par de estados consecutivos satisfaz a relação de transição individual.
- Em cada transição é selecionada a célula indicada na sequência.
- O último estado satisfaz o predicado de erro.

Se o resultado for `sat`, a sequência é um contraexemplo válido no modelo formal da FSM.

In [ ]:
```
def verificar_contraexemplo_z3(n, sequencia):
    estados = [
        criar_estado_z3(n, f"erro_{t}")
        for t in range(len(sequencia) + 1)
    ]

    solver = z3.Solver()

    # Estado inicial
    solver.add(predicado_inicial(estados[0], n))

    # Fronteira inicial morta
    for celula in borda_inicial_z3.values():
        solver.add(z3.Not(celula))

    # Transições individuais
    for t, (i, j) in enumerate(sequencia):
        indice = (i - 1) * n + (j - 1)

        solver.add(
            transicao_z3(
                estados[t],
                estados[t + 1],
                borda_inicial_z3,
                borda_final_z3,
```

```
                n,
                "individual",
                z3.IntVal(indice)
            )
        )

    # Estado final de erro
    solver.add(predicado_erro(estados[-1]))

    return solver.check()


resultado_erro_individual = verificar_contraexemplo_z3(
    N, sequencia_erro_individual
)

print("Resultado Z3:", resultado_erro_individual)

assert resultado_erro_individual == z3.sat

print("Contraexemplo confirmado pela FSM.")
```

Contraexemplo — Atualização simultânea

Na atualização simultânea, todas as células normais evoluem ao mesmo tempo, utilizando os valores da configuração anterior.

Para procurar um contraexemplo, considera-se uma configuração inicial válida com `N = 17` e valores de fronteira gerados aleatoriamente através da semente `97`.

Esta configuração permite alcançar o estado de erro após 3 transições:

- Estado inicial: 9 células vivas;
- Após 1 transição: 3 células vivas;
- Após 2 transições: 1 célula viva;
- Após 3 transições: 0 células vivas.

A sequência será inicialmente reproduzida em Python e posteriormente verificada com Z3.

A utilização de uma semente fixa permite reproduzir o contraexemplo, mas não limita a validade da conclusão: basta existir uma configuração inicial permitida que alcance o estado de erro para refutar a propriedade.

In [ ]:
```
def simular_simultanea(n, fronteira, finais, passos):
    # Construir a grelha inicial
    atual = [
        [1 if i < 3 and j < 3 else 0 for j in range(n)]
        for i in range(n)
    ]

    configuracoes = [atual]
```

```
    for _ in range(passos):
        # Todas as células usam o mesmo estado anterior
        seguinte = [
            [
                regra_b3_s23(
                    atual[i - 1][j - 1],
                    contar_vizinhos_vivos(
                        i, j, atual, fronteira, finais
                    )
                )
                for j in range(1, n + 1)
            ]
            for i in range(1, n + 1)
        ]

        atual = seguinte
        configuracoes.append(atual)

    return configuracoes
```

Teste do contraexemplo simultâneo

A função `simular_simultanea` é testada com a configuração de fronteira produzida pela semente `97`.

São executadas três transições e é contado o número de células normais vivas em cada estado.

Espera-se obter a sequência `[9, 3, 1, 0]`, confirmando que a última configuração corresponde ao estado de erro.

In [ ]:
```
_rng_97 = random.Random(97)

fronteira_97 = {
    pos: _rng_97.randint(0, 1)
    for pos in sorted(iniciais)
}

configuracoes_97 = simular_simultanea(
    N, fronteira_97, finais, 3
)

contagens_97 = [
    sum(sum(linha) for linha in estado)
    for estado in configuracoes_97
]

print("Células vivas:", contagens_97)

assert contagens_97 == [9, 3, 1, 0]

print("Contraexemplo simultâneo validado.")
```

Verificação simbólica com BMC

Para confirmar o contraexemplo, utiliza-se Bounded Model Checking (BMC) com profundidade `k = 3`.

São criados quatro estados simbólicos, correspondentes ao estado inicial e aos três estados seguintes.

O solver recebe as restrições:

- O primeiro estado satisfaz o predicado inicial.
- As células iniciais da fronteira assumem os valores gerados pela semente `97`.
- Cada par de estados consecutivos respeita a relação de transição simultânea.
- O último estado satisfaz o predicado de erro.

Um resultado `sat` demonstra a existência de uma execução válida que alcança o estado de erro.

Neste caso, é suficiente encontrar um contraexemplo para concluir que a propriedade de segurança é falsa.

In [ ]:
```python
# Código desenvolvido com apoio de LLM.
# Implementação revista e validada através dos testes do notebook.

def verificar_erro_simultaneo_z3(n, fronteira, passos):
    estados = [
        criar_estado_z3(n, f"sync97_{t}")
        for t in range(passos + 1)
    ]

    solver = z3.Solver()

    # Estado inicial
    solver.add(
        predicado_inicial(estados[0], n)
    )

    # Fixar a configuração da fronteira
    for pos, valor in fronteira.items():
        solver.add(
            borda_inicial_z3[pos] == z3.BoolVal(valor == 1)
        )

    # Construir as transições simultâneas
    for t in range(passos):
        solver.add(
            transicao_z3(
                estados[t],
                estados[t + 1],
                borda_inicial_z3,
                borda_final_z3,
```

```
                n,
                "simultanea"
            )
        )

    # Procurar o estado de erro
    solver.add(
        predicado_erro(estados[-1])
    )

    resultado = solver.check()

    if resultado == z3.sat:
        modelo = solver.model()

        contagens = [
            sum(
                z3.is_true(
                    modelo.eval(
                        celula,
                        model_completion=True
                    )
                )
                for celula in estado.values()
            )
            for estado in estados
        ]

        return resultado, contagens

    return resultado, None
```

Teste da verificação simbólica

O BMC é executado para a configuração de fronteira previamente utilizada, com profundidade `k = 3`.

Espera-se que o Z3 devolva `sat`, confirmando que existe uma sequência válida de transições que termina no estado de erro.

O número de células vivas em cada estado do modelo obtido pelo Z3 é também comparado com os resultados da simulação Python.

In [ ]:
```
resultado_sync, contagens_sync = verificar_erro_simultaneo_z3(
    N, fronteira_97, 3
)

print("Resultado Z3:", resultado_sync)
print("Células vivas por estado:", contagens_sync)

assert resultado_sync == z3.sat
assert contagens_sync == [9, 3, 1, 0]
```

```
print("Contraexemplo simultâneo confirmado pela FSM.")
```

Conclusão da propriedade de segurança

A propriedade de segurança estabelece que o autómato nunca deve alcançar um estado em que todas as células normais estão mortas.

Foram encontrados contraexemplos para ambas as modalidades:

Modalidade	Transições até ao erro	Resultado
Individual	9	Propriedade falsa
Simultânea	3	Propriedade falsa

Na atualização individual, o contraexemplo foi construído selecionando sucessivamente as 9 células inicialmente vivas, com a fronteira inicial completamente morta.

Na atualização simultânea, foi utilizada uma configuração de fronteira reproduzível através da semente `97`.

Os contraexemplos foram reproduzidos por simulação e verificados simbolicamente com Z3.

Assim, conclui-se que a propriedade **"Nunca se alcança o estado de erro"** não se verifica em nenhuma das duas modalidades consideradas.

B2 — Permanência das células

Atualização individual

Na modalidade individual, apenas uma célula normal é atualizada em cada transição, podendo manter o seu valor anterior.

Considera-se a célula `(8, 8)`, inicialmente morta e sem vizinhos vivos. A sua atualização mantém-na morta, enquanto todas as restantes células permanecem inalteradas.

Como a mesma célula pode ser selecionada em todas as transições, é possível construir uma execução infinita em que a configuração completa do autómato permanece constante.

Nesta execução, existem células permanentemente vivas e células permanentemente mortas.

Para confirmar este comportamento, utiliza-se o Z3 para verificar que a atualização selecionada não permite modificar o estado inicial.

In [ ]:
```
# Selecionar a célula (8, 8)
_indice_b2 = (8 - 1) * N + (8 - 1)
```

```
_teste_individual_b2 = z3.Solver()

_teste_individual_b2.add(
    predicado_inicial(estado0, N)
)

_teste_individual_b2.add(
    transicao_z3(
        estado0,
        estado1,
        borda_inicial_z3,
        borda_final_z3,
        N,
        "individual",
        z3.IntVal(_indice_b2)
    )
)

# Confirmar que a transição é possível
assert _teste_individual_b2.check() == z3.sat

# Procurar uma célula que tenha mudado de valor
_teste_individual_b2.push()

_teste_individual_b2.add(
    z3.Or(*[
        estado0[pos] != estado1[pos]
        for pos in estado0
    ])
)

assert _teste_individual_b2.check() == z3.unsat

_teste_individual_b2.pop()

print("Estado inicial mantém-se inalterado.")
print("Contraexemplo individual validado.")
```

Atualização simultânea

Para analisar a permanência das células na atualização simultânea, considera-se uma configuração em que todas as células iniciais da fronteira estão mortas.

Esta configuração é válida, uma vez que cada célula inicial pode assumir o valor `0` ou `1`.

A evolução é simulada durante 5 transições, utilizando a regra B3/S23.

Verifica-se que, após 4 transições, o autómato atinge um ponto fixo, isto é, um estado cuja atualização produz exatamente a mesma configuração.

Neste estado existem células vivas e mortas. Como a configuração não volta a mudar, estas células permanecem definitivamente nos respetivos estados.

In [ ]:
```python
# Fronteira inicial completamente morta
fronteira_zero_b2 = {
    pos: 0 for pos in iniciais
}

# Simular 5 transições simultâneas
configuracoes_b2 = simular_simultanea(
    N,
    fronteira_zero_b2,
    finais,
    5
)

# Contar células vivas
contagens_b2 = [
    sum(sum(linha) for linha in estado)
    for estado in configuracoes_b2
]

print("Células vivas:", contagens_b2)

# Verificar o resultado
assert contagens_b2 == [9, 6, 5, 7, 7, 7]

# Verificar que o estado 4 é igual ao estado 5
assert configuracoes_b2[4] == configuracoes_b2[5]

configuracao_fixa_b2 = configuracoes_b2[4]

# Identificar as células que permanecem vivas
posicoes_vivas_b2 = [
    (i + 1, j + 1)
    for i in range(N)
    for j in range(N)
    if configuracao_fixa_b2[i][j] == 1
]

print("Células vivas no ponto fixo:", posicoes_vivas_b2)
print("Ponto fixo encontrado.")
```

Verificação simbólica do ponto fixo

O ponto fixo encontrado na simulação é verificado através do Z3.

São criados cinco estados simbólicos, correspondentes ao estado inicial e às quatro transições necessárias para alcançar a configuração estável.

As restrições garantem que:

- O primeiro estado satisfaz o predicado inicial.
- Todas as células da fronteira inicial estão mortas.
- As quatro transições respeitam a atualização simultânea.
- O quarto estado corresponde à configuração estável encontrada.

Após confirmar a alcançabilidade, aplica-se novamente a relação de transição e procura-se uma célula que mude de valor.

Se esta última condição for insatisfazível, confirma-se que o estado é um ponto fixo da FSM.

In [ ]:
```python
# Código desenvolvido com apoio de LLM.
# Implementação revista e validada através dos testes do notebook.

def verificar_ponto_fixo_z3(n, configuracao_fixa):
    estados = [
        criar_estado_z3(n, f"b2_fixo_{t}")
        for t in range(5)
    ]

    solver = z3.Solver()

    # Estado inicial
    solver.add(predicado_inicial(estados[0], n))

    # Todas as células iniciais da fronteira mortas
    for celula in borda_inicial_z3.values():
        solver.add(z3.Not(celula))

    # Quatro transições simultâneas
    for t in range(4):
        solver.add(
            transicao_z3(
                estados[t],
                estados[t + 1],
                borda_inicial_z3,
                borda_final_z3,
                n,
                "simultanea"
            )
        )

    # Fixar o estado 4 à configuração encontrada
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            solver.add(
                estados[4][(i, j)] ==
                z3.BoolVal(
                    configuracao_fixa[i - 1][j - 1] == 1
                )
            )

    # Verificar alcançabilidade
    resultado_alcancabilidade = solver.check()

    if resultado_alcancabilidade != z3.sat:
        return resultado_alcancabilidade, None

    # Criar um estado seguinte
    seguinte = criar_estado_z3(n, "b2_fixo_seguinte")
```

```
    solver.push()

    solver.add(
        transicao_z3(
            estados[4],
            seguinte,
            borda_inicial_z3,
            borda_final_z3,
            n,
            "simultanea"
        )
    )

    # Tentar encontrar alguma célula que mude
    solver.add(
        z3.Or(*[
            estados[4][pos] != seguinte[pos]
            for pos in estados[4]
        ])
    )

    resultado_alteracao = solver.check()

    solver.pop()

    return resultado_alcancabilidade, resultado_alteracao
```

Teste da verificação simbólica

O Z3 é utilizado para confirmar a alcançabilidade e a estabilidade da configuração encontrada.

Espera-se que a alcançabilidade devolva `sat`, demonstrando que o ponto fixo pode ser alcançado a partir de um estado inicial permitido.

Em seguida, a tentativa de produzir um estado diferente deve devolver `unsat`, demonstrando que a configuração não se altera na transição seguinte.

Como o estado é estável e a evolução simultânea é determinística, a configuração mantém-se indefinidamente.

In [ ]:
```
resultado_alcance_b2, resultado_mudanca_b2 = (
    verificar_ponto_fixo_z3(
        N,
        configuracao_fixa_b2
    )
)

print("Alcançabilidade:", resultado_alcance_b2)
print("Possibilidade de mudança:", resultado_mudanca_b2)

assert resultado_alcance_b2 == z3.sat
```

```python
assert resultado_mudanca_b2 == z3.unsat

print("Ponto fixo confirmado pela FSM.")
```

Conclusão da propriedade de permanência

A propriedade estabelece que nenhuma célula normal deve permanecer definitivamente morta ou viva.

Foram encontrados contraexemplos para ambas as modalidades de transição.

Atualização individual

Foi demonstrada a existência de uma execução infinita em que o estado inicial permanece inalterado, selecionando repetidamente a célula `(8, 8)`.

Nesta execução, as 9 células inicialmente vivas permanecem vivas e as restantes continuam mortas.

Atualização simultânea

Com a fronteira inicial completamente morta, o autómato alcança um ponto fixo após 4 transições.

Nesse estado, existem 7 células permanentemente vivas e as restantes 282 células normais permanecem mortas.

A alcançabilidade e a estabilidade desta configuração foram verificadas através do Z3.

Conclui-se que a propriedade é falsa em ambas as modalidades, uma vez que existem execuções válidas nas quais algumas células permanecem definitivamente vivas ou mortas.

Testes de correção

Para validar a implementação da FSM, são realizados testes que comparam a evolução calculada diretamente em Python com a evolução representada simbolicamente em Z3.

São consideradas as duas modalidades de transição:

- **Individual:** atualização da célula `(2, 2)`, mantendo todas as restantes inalteradas.
- **Simultânea:** atualização de todas as células normais.

Em ambos os casos é utilizada a configuração inicial com `N = 17` e a fronteira inicial completamente morta.

O objetivo é verificar que todos os valores do estado seguinte calculados em Python coincidem com os valores impostos pela relação de transição em Z3.

Esta comparação complementa os testes anteriores, permitindo detetar possíveis diferenças entre a implementação operacional e o modelo lógico.

In [ ]:
```python
# Código desenvolvido com apoio de LLM.
# Implementação revista e validada através dos testes do notebook.

def validar_transicao_z3(n, modo):
    # Estado inicial concreto
    grelha_teste = [
        [1 if i < 3 and j < 3 else 0 for j in range(n)]
        for i in range(n)
    ]

    fronteira_teste = {
        pos: 0 for pos in borda_inicial_z3
    }

    # Calcular a evolução através de Python
    if modo == "simultanea":
        seguinte_python = simular_simultanea(
            n, fronteira_teste, finais, 1
        )[1]

    elif modo == "individual":
        seguinte_python = [
            linha.copy() for linha in grelha_teste
        ]

        vivos = contar_vizinhos_vivos(
            2, 2, grelha_teste, fronteira_teste, finais
        )

        seguinte_python[1][1] = regra_b3_s23(
            grelha_teste[1][1], vivos
        )

    else:
        raise ValueError("Modalidade inválida")

    # Criar os estados simbólicos
    antes = criar_estado_z3(n, f"validar_{modo}_0")
    depois = criar_estado_z3(n, f"validar_{modo}_1")

    solver = z3.Solver()
    solver.add(predicado_inicial(antes, n))

    # Fixar a fronteira a zero
    for celula in borda_inicial_z3.values():
        solver.add(z3.Not(celula))

    # Selecionar a modalidade
    if modo == "individual":
        indice = (2 - 1) * n + (2 - 1)

        transicao = transicao_z3(
```

```
            antes, depois,
            borda_inicial_z3, borda_final_z3,
            n, modo, z3.IntVal(indice)
        )
    else:
        transicao = transicao_z3(
            antes, depois,
            borda_inicial_z3, borda_final_z3,
            n, modo
        )

    solver.add(transicao)

    assert solver.check() == z3.sat

    # Procurar alguma diferença relativamente ao Python
    diferencas = [
        depois[(i, j)] != z3.BoolVal(
            seguinte_python[i - 1][j - 1] == 1
        )
        for i in range(1, n + 1)
        for j in range(1, n + 1)
    ]

    solver.add(z3.Or(*diferencas))

    return solver.check() == z3.unsat
```

Validação das duas modalidades

A função de teste é aplicada às duas modalidades de atualização.

Para cada modalidade, o Z3 procura uma diferença entre o estado seguinte calculado por Python e o estado seguinte definido pela FSM.

Espera-se que ambas as verificações devolvam `True`, indicando que não existem diferenças possíveis para as configurações analisadas.

In [ ]:
```
assert validar_transicao_z3(N, "individual")
print("Atualização individual validada.")

assert validar_transicao_z3(N, "simultanea")
print("Atualização simultânea validada.")

print("Testes de correção concluídos.")
```

Eficiência computacional e escalabilidade

Para avaliar a eficiência da implementação, são realizados testes com diferentes dimensões da grelha:

- `N = 17` : 289 células normais;
- `N = 20` : 400 células normais;
- `N = 25` : 625 células normais.

São analisadas as duas modalidades de transição, individual e simultânea.

Para permitir uma comparação consistente, todos os testes utilizam o mesmo padrão de estado inicial e células de fronteira mortas.

São medidos dois tempos:

- **Construção:** tempo necessário para criar as variáveis simbólicas e construir as restrições da FSM.
- **Resolução:** tempo necessário para o Z3 verificar a satisfabilidade de uma transição.

Cada configuração é executada três vezes, sendo calculada a média dos tempos obtidos.

Esta análise permite observar o crescimento do custo computacional à medida que aumenta o número de células.

In [ ]:
```python
import time

def medir_instancia_conway(n, modo):
    inicio = time.perf_counter()

    # Criar estados simbólicos
    antes = criar_estado_z3(n, f"escala_{n}_{modo}_0")
    depois = criar_estado_z3(n, f"escala_{n}_{modo}_1")

    # Criar células iniciais da fronteira
    borda_inicial = {
        (i, j): z3.Bool(f"escala_b_{n}_{modo}_{i}_{j}")
        for i in range(n + 1)
        for j in range(n + 1)
        if i == 0 or j == 0
    }

    # Criar células finais
    borda_final = {
        (i, j): z3.BoolVal(False)
        for i in range(n + 2)
        for j in range(n + 2)
        if i == n + 1 or j == n + 1
    }

    solver = z3.Solver()

    solver.add(predicado_inicial(antes, n))

    # Fixar a fronteira inicial a zero
    for celula in borda_inicial.values():
        solver.add(z3.Not(celula))
```

```
    # Construir a relação de transição
    if modo == "individual":
        indice = (8 - 1) * n + (8 - 1)
        escolhida = z3.IntVal(indice)
    else:
        escolhida = None

    solver.add(
        transicao_z3(
            antes, depois,
            borda_inicial, borda_final,
            n, modo, escolhida
        )
    )

    fim_construcao = time.perf_counter()

    resultado = solver.check()

    fim_resolucao = time.perf_counter()

    if resultado != z3.sat:
        raise RuntimeError(
            f"Resultado inesperado: {resultado}"
        )

    return (
        fim_construcao - inicio,
        fim_resolucao - fim_construcao
    )
```

Testes de desempenho

A função `medir_instancia_conway` é executada para as dimensões `17`, `20` e `25`, considerando ambas as modalidades.

Para cada combinação são realizadas três execuções independentes, calculando-se a média dos tempos de construção e resolução.

Na atualização individual, é selecionada a célula `(8, 8)`.

Os testes analisam apenas uma transição, não incluindo a exploração de múltiplos estados através de BMC. Desta forma, os resultados medem o custo de construção e satisfabilidade de uma transição simbólica, não o custo de provar propriedades temporais ilimitadas.

In [ ]:
```
resultados_escala = []

for _n in [17, 20, 25]:
    for _modo in ["individual", "simultanea"]:

        _construcoes = []
        _resolucoes = []
```

```
        for _ in range(3):
            _tc, _tr = medir_instancia_conway(
                _n, _modo
            )

            _construcoes.append(_tc)
            _resolucoes.append(_tr)

        _media_construcao = sum(_construcoes) / 3
        _media_resolucao = sum(_resolucoes) / 3

        resultados_escala.append({
            "N": _n,
            "modo": _modo,
            "construcao": _media_construcao,
            "resolucao": _media_resolucao
        })

        print(
            f"N={_n} | {_modo} | "
            f"construção={_media_construcao:.4f}s | "
            f"resolução={_media_resolucao:.4f}s"
        )
```

Resultados dos testes de escalabilidade

Tempo de construção das restrições (segundos)

N	Modalidade	Execução 1	Execução 2	Execução 3	Tempo médio
17	Individual	0.2115	0.2010	0.2014	0.2046
17	Simultânea	0.1860	0.1833	0.1858	0.1850
20	Individual	0.2733	0.2719	0.2723	0.2725
20	Simultânea	0.2526	0.2537	0.2523	0.2529
25	Individual	0.4266	0.4246	0.4263	0.4258
25	Simultânea	0.3992	0.4171	0.3957	0.4040

Tempo de resolução do Z3 (segundos)

N	Modalidade	Execução 1	Execução 2	Execução 3	Tempo médio
17	Individual	0.0025	0.0022	0.0022	0.0023
17	Simultânea	0.0034	0.0032	0.0033	0.0033
20	Individual	0.0027	0.0027	0.0027	0.0027
20	Simultânea	0.0043	0.0044	0.0044	0.0044
25	Individual	0.0037	0.0037	0.0037	0.0037

N	Modalidade	Execução 1	Execução 2	Execução 3	Tempo médio
25	Simultânea	0.0062	0.0063	0.0063	0.0063

Análise dos resultados

Os resultados mostram que o tempo de construção das restrições aumenta com a dimensão da grelha, enquanto o tempo de resolução do Z3 permanece relativamente reduzido nas instâncias testadas.

Para `N = 17`, a construção demora aproximadamente 0.20 segundos na modalidade individual e 0.19 segundos na simultânea. Para `N = 25`, estes valores aumentam para aproximadamente 0.43 e 0.40 segundos, respetivamente.

A construção das restrições representa a maior parte do tempo de execução, correspondendo a cerca de 98% a 99% do tempo total.

A modalidade individual apresenta tempos de construção ligeiramente superiores, uma vez que necessita de condições adicionais para selecionar a célula atualizada e manter as restantes inalteradas.

Por outro lado, o tempo de resolução da modalidade simultânea é superior, porque o Z3 tem de considerar a aplicação da regra B3/S23 a todas as células normais.

Apesar do aumento do número de células, os tempos de resolução permanecem inferiores a 0.01 segundos nos testes realizados.

Estes resultados mostram que a implementação permite construir e resolver simbolicamente transições para diferentes dimensões, sem alterações à estrutura do modelo.

Contudo, os testes representam apenas uma transição satisfazível. Não permitem avaliar diretamente o custo da verificação de propriedades temporais através de múltiplas transições, nem demonstrar uma complexidade assintótica específica.

Decisões de implementação

Representação dos estados

As células normais foram representadas por variáveis booleanas do Z3, uma vez que cada célula pode assumir apenas dois estados: viva ou morta.

As coordenadas das células são utilizadas como chaves de dicionários, permitindo aceder diretamente às variáveis e identificar os seus vizinhos.

A dimensão da grelha é parametrizável através de `N`, permitindo utilizar o mesmo modelo para diferentes dimensões.

Células de fronteira

As células iniciais da fronteira são representadas por variáveis booleanas simbólicas, permitindo considerar diferentes configurações iniciais.

Para realizar simulações reproduzíveis, são também geradas configurações concretas através de sementes aleatórias.

As células finais são representadas pela constante `False`, uma vez que permanecem sempre mortas.

Modalidades de transição

Foram implementadas duas modalidades de transição, selecionadas através do parâmetro `modo`.

Na modalidade individual, é selecionada exatamente uma célula para aplicar a regra B3/S23. A atualização pode manter o valor da célula, caso a regra determine que esta permanece viva ou morta.

Na modalidade simultânea, todas as células normais são atualizadas utilizando exclusivamente os valores do estado anterior.

Ambas as modalidades utilizam a mesma função simbólica para representar a regra B3/S23, evitando duplicação da lógica de evolução.

Utilização do Z3

Foi utilizado o solver SMT Z3 para representar simbolicamente a FSM e verificar as propriedades pedidas.

A verificação de contraexemplos foi realizada através de restrições sobre estados e relações de transição.

Para a propriedade de segurança, foram construídas sequências válidas que alcançam o estado de erro.

Para a propriedade de permanência das células, foram identificadas execuções que alcançam ou preservam configurações estáveis, permitindo justificar comportamentos infinitos.

Testes e validação

A implementação foi validada através de testes automáticos das funções de vizinhança, da regra B3/S23, das condições iniciais e das relações de transição.

Foram também comparados os resultados da simulação Python com os resultados obtidos através do modelo Z3.

Finalmente, foram realizados testes de desempenho para diferentes dimensões da grelha, medindo separadamente os tempos de construção e resolução.

Contribuição da LLM

Durante o desenvolvimento deste trabalho foi utilizada uma ferramenta LLM (ChatGPT) como apoio à compreensão do problema, à implementação da solução e à validação dos resultados.

A utilização da LLM incidiu principalmente nas seguintes tarefas:

- interpretação dos requisitos do exercício e organização do notebook Marimo;
- definição dos estados, das células de fronteira e da vizinhança;
- implementação da regra B3/S23;
- apoio na construção da FSM e das relações de transição utilizando Z3;
- discussão da diferença entre atualização individual e simultânea;
- construção e verificação de contraexemplos para as propriedades de segurança e permanência;
- desenvolvimento do código;
- correção de problemas de implementação, incluindo a contagem das células finais e conflitos de variáveis no Marimo;
- organização dos testes de eficiência computacional e escalabilidade;
- apoio na documentação e apresentação dos resultados.

Registo da interação

Ligação à conversa utilizada no desenvolvimento:

Registo da interação com a LLM

Conclusão

Neste trabalho foi desenvolvido um modelo do Conway's Game of Life modificado, utilizando Python e o solver SMT Z3 para representar uma Máquina de Estados Finita (FSM).

Foram implementadas as duas modalidades de transição previstas no enunciado: atualização individual e atualização simultânea, ambas baseadas na regra B3/S23.

Verificação das propriedades

Na propriedade de segurança (B1), foram encontrados contraexemplos para ambas as modalidades. Na atualização individual, o estado de erro foi alcançado após 9 transições. Na

atualização simultânea, uma configuração inicial válida permitiu alcançar o mesmo estado após 3 transições.

Na propriedade de permanência (B2), verificou-se que também existem execuções em que algumas células permanecem definitivamente vivas ou mortas. Na modalidade individual, foi construída uma execução infinita que mantém o estado inicial. Na modalidade simultânea, foi identificado um ponto fixo alcançado após 4 transições.

Assim, ambas as propriedades foram refutadas nas duas modalidades consideradas.

Eficiência computacional

Os testes de escalabilidade foram realizados para `N = 17`, `N = 20` e `N = 25`.

Os resultados mostraram que o tempo de construção das restrições aumenta com a dimensão da grelha, enquanto o tempo de resolução de uma única transição pelo Z3 permaneceu inferior a 0.01 segundos nas configurações testadas.

Considerações finais

A abordagem permitiu representar formalmente a evolução do autómato e obter contraexemplos verificáveis para as propriedades pedidas.

A comparação entre simulações Python e restrições Z3 contribuiu para validar a consistência da implementação.

Como limitação, os testes de escalabilidade consideraram apenas uma transição simbólica, não permitindo caracterizar o custo de verificações temporais com maior profundidade.